

CSE 256 WI26 Midterm Review with Solutions - (75 Points)

Part 0: Instructions

- You are allowed one double-sided A4 note sheet.
- You are not allowed calculators or other electronic devices.
- Partial credit will be given for short-answer and long-answer questions, so it is to your advantage to show work in the exam.
- **The penalty for academic dishonesty will be a course grade of F and a referral of the case to UCSD Academic Integrity Office. Further penalties, including suspension or expulsion from the University may be imposed by that office.**
- By turning in this exam with your name on it, you are certifying that this is yours and yours alone. You must not copy from others, and you must not allow others to copy from you.

Name: _____

Student ID: _____

Part 1: True/False Questions (14 points)

1. Compared to n -gram language models, feed-forward neural network language models handle out-of-vocabulary words naturally.
☐ True
☒ False
2. Word embeddings like Word2Vec and GloVe can be learned from unlabeled text data.
☒ True
☐ False
3. $\tanh$ can be used as an activation function in neural networks.
☒ True
☐ False
4. SGD can be applied to only convex functions.
☐ True
☒ False
5. We can inspect unlabeled test data to inform our design choices during model development as long as the model does not have access to the test data.
☐ True
☒ False
6. In a language model, higher perplexity on the held-out data indicates a better model.
☐ True

✓ False

7. The bag-of-words representation of a sentence is a vector of dimension d where d is the number of unique words in the sentence.

☐ True

✓ False

8. Recurrent neural network language models use longer context than typical n-gram language models.

✓ True

☐ False

9. The softmax function is commonly used in the output layer of transformers for classification tasks.

✓ True

☐ False

10. A GPT model is an encoder-only model used for tasks like sentence classification due to their use of bidirectional context information.

☐ True

✓ False

11. Positional encodings in transformers are necessary to capture sequence order information, which the architecture itself cannot inherently handle.

✓ True

☐ False

12. Suppose you remove the nonlinearity from the standard Feedforward Neural Networks, any classification model represented by this neural network can be represented with a softmax classifier.

✓ True

☐ False

13. Beam search with a beam width of 1 is equivalent to greedy decoding.

✓ True

☐ False

14. In multi-head self-attention, each attention head independently computes its own attention weights, allowing the model to capture different types of relationships in the data.

✓ True

☐ False

Part 2: Multiple Choice Questions (46 points)

1. (2 points) Suppose we want a system to take as input some text $\mathbf{x}$ (sentence, document, etc.), then output a label y from a set of five class labels $\mathcal{Y}$. What is the most correct name for this type of machine learning problem?

✓ Classification

☐ Language modeling

☐ Sequence to Sequence

☐ Question Answering

2. (3 points) You have trained a softmax classifier and observed overfitting on the validation set. What should you do to mitigate the problem? Select all that apply.
- ☐ Collect more test data
 - ✓ Add a regularization term to the loss
 - ✓ Collect more training data
 - ☐ Collect more validation data
3. (3 points) Which of the following are true about the softmax function? Select all that apply.
- ✓ Softmax is used to convert raw scores into probabilities.
 - ✓ Softmax is differentiable.
 - ☐ Softmax can produce negative probability values if the inputs to the softmax are negative.
 - ☐ Softmax classifier is only defined for two-class (binary) classification problems.
4. (2 points) Which of the following is NOT a concept directly involved in parameter learning?
- ☐ Backpropagation
 - ☐ Stochastic gradient descent
 - ✓ Beam search
 - ☐ Maximum likelihood estimation

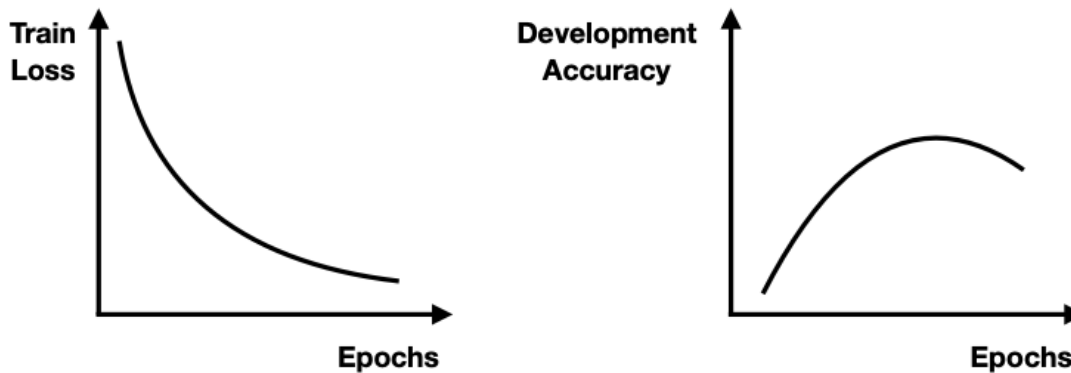


Figure 1:

5. (3 points) In Figure 1 above, we see train loss and validation accuracy plotted for a model. Which of the following statements are true? Select all that apply.
- ☐ The model is underfitting the training dataset
 - ✓ The model is overfitting the training dataset
 - ☐ Training the model for even more epochs would likely improve the development accuracy
 - ✓ Add regularization, such as dropout, could lead to better development accuracy.
6. (3 points) (3 points) You want to use word embeddings and a FFNN (feed-forward neural network) to predict positive or negative sentiment of a sentence. Which of the following is the best implementation of this FFNN assuming that other code around it is written correctly? The brackets $[input \times output]$ indicate the dimensions of each transformation.

- ☐ 300-dim embedding layer $\rightarrow$ $[300 \times 50]$ Linear layer $\rightarrow$ Tanh $\rightarrow$ $[100 \times 2]$ Linear $\rightarrow$ softmax
 - ☐ 50-dim embedding layer $\rightarrow$ $[50 \times 150]$ Linear layer $\rightarrow$ $[150 \times 2]$ Linear layer $\rightarrow$ softmax
 - ☒ 50-dim embedding layer $\rightarrow$ $[50 \times 75]$ Linear layer $\rightarrow$ Tanh $\rightarrow$ $[75 \times 2]$ Linear $\rightarrow$ log-softmax
 - ☐ 300-dim embedding layer $\rightarrow$ $[300 \times 200]$ Linear layer $\rightarrow$ Tanh $\rightarrow$ $[200 \times 2]$ Linear $\rightarrow$ Tanh $\rightarrow$ softmax
7. (4 points) You developed a sentiment analysis system that feeds word tokens into a unidirectional RNN, then outputs the sentiment class by putting the final hidden state through a linear + softmax layer. Now you want to try other approaches to compare to on this task, what other approaches can tackle this task. Select all that apply.
- ☒ Deep averaging network
 - ☒ Transformer
 - ☒ LSTM
 - ☒ Bag of Words (BOW) Model
8. (2 points) Which of the following would be unnecessary or irrelevant in a Deep Averaging Network?. Select all that apply.
- ☒ Positional embeddings for word order
 - ☒ Self-attention parameters
 - ☐ Feedforward layer weights
 - ☐ Embedding layer for input tokens
9. (4 points) You train a sentence-level classification model (e.g., for sentiment analysis) on a dataset containing only present-tense sentences. You then apply it to a test set that includes past and future tenses. Which of the following could help the model generalize better? Select all that apply.
- ☒ Using pre-trained word embeddings
 - ☒ Find training data with all tenses represented
 - ☐ Find validation data with all tenses represented
 - ☒ Use subword tokenization
10. (3 points) You are training a language model (as in PA3), which of the following is a possible correct shape for the attention map? Select all that apply.
- ☐ [batch size, seq len]
 - ☐ [seq len, vocab size]
 - ☐ [batch size, seq len, d_{model}]
 - ☒ [batch size, seq len, seq len]
11. (3 points) You are training a language model as in PA3, which of the following is a possible correct shape for the model outputs that are input to the loss function? Select all that apply.
- ☐ [batch size, seq len]
 - ☐ [batch size, seq len, seq len]
 - ☐ [batch size, seq len, d_{model}]
 - ☒ [batch size, seq len, vocab size]

12. (4 points) Which of the following would be most likely to decrease the accuracy of a pre-trained BERT model when fine-tuned on downstream tasks? Select all that apply.
- ☒ Setting the MASK percentage to 0% instead of the usual 15% during pre-training
 - ☒ Decreasing the hidden dimension of the BERT model
 - ☒ Pre-training BERT on a smaller dataset
 - ☐ Pre-training BERT on a larger dataset
13. (2 points) You remove one of the attention heads in a multi-head attention mechanism, how does this change affect the total number of parameters in the query, key, and value matrices?
- ☐ The number of parameters increases because fewer heads mean more weights per head.
 - ☒ The number of parameters decreases because each head has its own set of query, key, and value matrices.
 - ☐ The number of parameters remains the same since the matrices are shared across heads.
 - ☐ The effect on parameters depends on the overall model size.
14. (2 points) In Transformer-based language models, a decoder uses a left-context-only approach, meaning that for a sequence of tokens $t_1, t_2, \dots, t_n$, the model can access only tokens t_1 through t_i when predicting t_{i+1} . How does this impact the masked self-attention used during training?
- ☐ The masked self-attention scores must be all ones, allowing access to all tokens in the sequence.
 - ☒ The masked self-attention scores are set to zero after the current token t_i , blocking access to any future tokens $t_{i+1}, t_{i+2}, \dots, t_n$.
 - ☐ Masked self-attention enables the model to access both past and future tokens during training.
 - ☐ Masked self-attention does not affect the tokens available to the decoder, as it is only relevant for the encoder.
15. (2 points) Which decoding strategy explores multiple possible sequences simultaneously and selects the highest-scoring sequence at the end of decoding?
- ☐ Greedy decoding
 - ☒ Beam search
 - ☐ Top-k sampling
 - ☐ Nucleus sampling
16. (2 points) In nucleus sampling, how is the next token chosen?
- ☐ The token with the highest probability is chosen, without considering other tokens.
 - ☒ The next token is chosen from a dynamically determined set of tokens with cumulative probabilities up to a threshold p (e.g., 0.9).
 - ☐ The next token is chosen randomly from a fixed top k of the most probable tokens.
 - ☐ The next token is chosen by scaling the probabilities with a temperature factor.
17. (2 points) When does the vanishing gradient problem occur in neural networks?
- ☐ When the gradient is too large
 - ☒ When the gradient is too small
 - ☐ When the gradient is zero
 - ☐ When the gradient is negative

Part 3: Explanation Questions (15 points)

Machine Translation is a sequence-to-sequence (seq-to-seq) problem: both the input and output are sequences (of words). A common neural architecture used for seq-to-seq problems is the encoder-decoder model, depicted in Figure 2. In its original formulation, the encoder-decoder model consists of two recurrent neural networks (RNNs). The encoder RNN compresses the input sequence into a fixed-length vector, represented by the final encoder hidden state. The decoder RNN conditions on this vector to produce the translation.

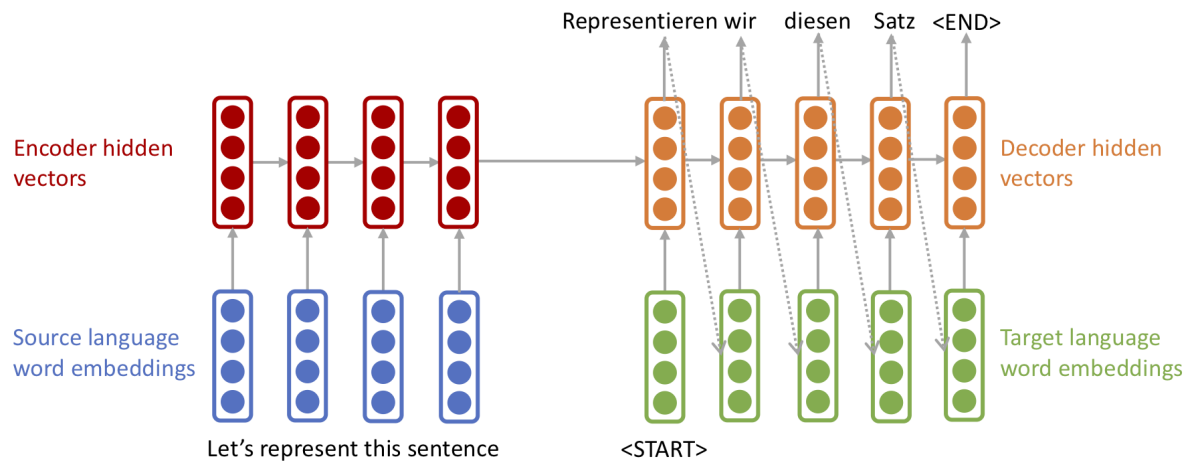


Figure 2: Encoder-Decoder architecture

1A) How do you think the architecture in Figure 2 will perform when translating long sentences, and why? Consider the amount of information the decoder gets to see about the input sequence. (3 points)

Answer: This vanilla encoder-decoder model will perform poorly on long sentences because the decoder only gets to see the final encoder hidden state, which may not contain enough information about the input sequence. The model may struggle to accurately translate long sentences due to the limited context provided to the decoder. There is an information bottleneck.

1B) What modification can we make to this architecture to improve its performance on long sentences, while still using RNNs for the encoder and decoder. Explain how this modification facilitates more accurate translation of long sequences. (4 points)

Answer: We can use cross-attention, where the decoder attends to the encoder's hidden states at each time step. This modification allows the decoder to access more information about the input sequence, improving its ability to translate long sentences.

Common mistake: saying that we can use an LSTM instead, but this does not address the issue of the decoder not having access to the full input sequence.

2. Today, Transformers have replaced RNNs in the encoder-decoder architectures. Transformers get rid of the recurrence that is central to RNNs. Instead, they rely completely on attention. In self-attention, each element in the sentence attends to other elements, resulting in context-sensitive representations.

2A) Transformers have enabled scaling to longer sequences, effectively encoding long-range dependencies and allowing models to condition on longer contexts in language modeling. Additionally, they support parallelization, which speeds up training and inference. How do Transformers achieve both of these capabilities? (4 points)

Answer:

1. **Long-range dependencies:** Transformers achieve this through self-attention, which allows every token in a sequence to attend to all other tokens. This mechanism enables the model to directly capture relationships between distant tokens, as each token has global access to the entire sequence.
2. **Parallelization:** Self-attention supports parallel processing by handling all tokens in a sequence simultaneously. Unlike RNNs, which process tokens sequentially due to their reliance on recurrence, Transformers compute attention weights and outputs for the entire sequence in parallel, significantly speeding up training and inference.

Common mistake: A common misconception is attributing parallelism to the presence of multiple attention heads. However, parallelism is inherent to the self-attention mechanism itself and does not depend on the number of heads. Even with a single attention head, the model processes the entire sequence simultaneously. Multi-head attention improves the model's ability to capture diverse relationships in the data while preserving parallelism.

2B) Recall that attention can be viewed as an operation on a query $q \in \mathbb{R}^d$, a set of value vectors $\{v_1, \dots, v_n\}, v_i \in \mathbb{R}^d$, and a set of key vectors $\{k_1, \dots, k_n\}, k_i \in \mathbb{R}^d$, specified as follows:

$$c = \sum_{i=1}^n v_i \alpha_i$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^n \exp(k_j^\top q)}$$

with α_i termed the “attention weights”. The output $c \in \mathbb{R}^d$ is an average over the value vectors weighted with respect to α_i .

Here the attention function that computes α_i , is the **dot product**, applied to the query and the key vector $(k_i^\top q)$. If we introduce a temperature parameter $\tau > 0$ such that

$$\alpha_i = \frac{\exp\left(\frac{k_i^\top q}{\tau}\right)}{\sum_{j=1}^n \exp\left(\frac{k_j^\top q}{\tau}\right)}$$

, analyze the effect of τ on the concentration of the attention weights. Specifically, discuss what happens to the attention distribution as $\tau \rightarrow 0$ and $\tau \rightarrow \infty$. (4 points)

Answer:

The parameter τ controls the sharpness or spread of the attention distribution:

1. **As $\tau \rightarrow 0$:** The attention distribution becomes highly **concentrated**, with a small number of key vectors receiving most of the attention. This is because dividing $k_i^\top q$ by a small τ exaggerates differences in dot products. The exponentials corresponding to the largest dot products dominate, leading to nearly one-hot attention weights.
2. **As $\tau \rightarrow \infty$:** The attention distribution becomes increasingly **uniform**, with all key vectors receiving similar attention. Dividing $k_i^\top q$ by a large τ reduces differences between dot products, causing all exponentials to approach 1. The normalization then assigns nearly equal weight to all keys.

The temperature τ adjusts the focus of the attention mechanism: smaller τ sharpens the focus on a few keys, while larger τ spreads the focus evenly across all keys.

Common error: **Confusing temperature in attention with randomness in decoding:** A common misconception is treating τ as introducing randomness, as it does in decoding during generation. Here, τ only modifies the **concentration** of the attention weights. It controls how sharply the weights focus on specific keys.

Bonus: 2 Points

Other than (scaled) dot product attention, name an alternative attention function for computing query-key similarities. (2 points)

Answer: additive attention, multiplicative attention, etc.